

Networks & System Security

SIL765 & COL7165 - Quiz 1 - 16 January 2026

Name: _____

Entry Number: _____

Select Course Code: ☐ SIL765 ☐ COL7165

Instructions:

- Each question is a **multiple-select** format.
- One, multiple, or all options may be correct.
- There will be no partial credit unless explicitly stated by the instructor.
- To indicate your choice, please place a check mark (✓) next to the selected option. No other symbols will be accepted.

Q1. Length Extension and HMAC

Suppose a system authenticates messages using:

$$\text{tag} = \text{SHA256}(\text{secret} \parallel \text{message}).$$

Which statements are correct?

- A) The scheme is vulnerable to length-extension attacks.
- B) An attacker can append extra data and forge a valid tag without knowing the secret.
- C) Replacing it with HMAC eliminates all vulnerabilities.
- D) Using SHA-512 instead of SHA-256 automatically fixes the problem.

• Correct options: A, B, C

- **Why:** This construction is a *prefix-MAC* using a *Merkle–Damgård* hash (SHA-256 is Merkle–Damgård style hash). In Merkle–Damgård, hashing proceeds block-by-block and the output is the final chaining value:

$$C_0 = IV, \quad C_{i+1} = f(C_i, m_i), \quad H(M) = C_n.$$

(Ref: 07-crypto hash)

- **A) Vulnerable to length extension — TRUE.** If an attacker knows $H(\text{secret} \parallel \text{message})$, they effectively learn the final chaining value after processing $\text{secret} \parallel \text{message}$. Since Merkle–Damgård continues iterating f over additional blocks, the attacker can compute

$$H(\text{secret} \parallel \text{message} \parallel \text{pad} \parallel \text{extra})$$

without knowing secret, as long as they can guess the secret length for correct padding.

- **B) Can append extra data + forge a valid tag — TRUE.** Using the length-extension computation, the attacker can generate a valid tag for a longer message ($\text{message} \parallel \text{extra}$).
- **C) Replace with HMAC removes this vulnerability — TRUE.** HMAC is standardized as:

$$\text{HMAC}(k, \text{msg}) = H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel \text{msg})).$$

Here the attacker never receives a tag that exposes a chaining value for $k \parallel \text{msg}$.

- **D) Using SHA-512 fixes it — FALSE.** SHA-512 is also Merkle–Damgård, so the structural reason length extension exists remains.

Q2. AES-NI and Side Channels

Which of the following are true about AES-NI?

- A) AES-NI rounds are implemented as CPU instructions.
- B) AES-NI removes all timing side-channel risks from the entire application.
- C) AES-NI provides significant speedup over software AES.
- D) AES-NI instructions are designed to be constant-time.

- **Correct options: A, C, D**
- **Why:** Slides state: **AES-NI = AES in hardware**, with instructions like `aesenc`, `aesdec`, and `aeskeygenassist`, giving *more than 10x speedup* and that *all AES instructions are constant time*.
(Ref: 07-crypto hash)
- **A) AES rounds are CPU instructions — TRUE.** The slide explicitly lists `aesenc/aesenclast` and `aesdec/aesdeclast` as instructions that implement AES rounds.
(Ref: 07-crypto hash)
- **B) Removes all timing side-channel risks in the entire app — FALSE.** The slide's constant-time claim applies to *AES-NI instructions* (i.e., the AES computation), not to the full application/protocol logic, memory accesses, I/O timing, or other code paths.
- **C) Significant speedup — TRUE.** The slide states *more than 10x speedup over a software AES*.
(Ref: 07-crypto hash)
- **D) Designed constant-time — TRUE.** The slide states *better security: all AES instructions are constant time*.
(Ref: 07-crypto hash)

Q3. Diffie–Hellman Threat Model

Which statements about Diffie–Hellman are correct?

- A) Diffie–Hellman allows key agreement over an insecure channel.
- B) Diffie–Hellman prevents man-in-the-middle attacks by default.
- C) Diffie–Hellman security relies on the discrete logarithm problem.
- D) Authentication must be added separately.

- **Correct options: A, C, D**
- **A) Key agreement over insecure channel — TRUE.** The primary purpose of Diffie–Hellman is to allow two parties to establish a shared secret over a public (insecure) communication channel.
- **B) Prevents man-in-the-middle by default — FALSE.** Diffie–Hellman by itself provides no authentication. An active attacker can intercept and replace public values, establishing separate keys with each party (man-in-the-middle attack), unless authentication mechanisms such as certificates or digital signatures are used.
- **C) Relies on discrete logarithm hardness — TRUE.** The security of Diffie–Hellman relies on the computational difficulty of solving the discrete logarithm problem in the chosen group.
- **D) Authentication must be added — TRUE.** To prevent man-in-the-middle attacks, Diffie–Hellman must be combined with authentication mechanisms such as public-key certificates, signatures, or pre-shared keys.

Q4. Encrypt/MAC Ordering

Which constructions provide provable authenticated encryption security?

- A) Encrypt-then-MAC

- B) MAC-then-Encrypt
- C) Encrypt-and-MAC
- D) MAC only

- **Correct option: A only**
- **Why:** The slide titled “Combining MAC and ENC (Auth. Enc.)” shows three constructions for combining encryption and message authentication and explicitly labels one of them as “*always correct*”.
- That *always correct* construction corresponds to **Encrypt-then-MAC**, where the message is first encrypted and the MAC is computed over the ciphertext:

$$c = \text{Enc}(k_E, m), \quad t = \text{MAC}(k_I, c).$$

(Ref: 07-crypto hash)

- **A) Encrypt-then-MAC — TRUE.** This matches the construction explicitly marked as *always correct* on the slide.
- **B) MAC-then-Encrypt — FALSE.** Although it has been used historically (e.g., in SSL/TLS), it is not universally secure across all modes and implementations.
- **C) Encrypt-and-MAC — FALSE.** This construction is used in some systems, but it is not labeled as *always correct* and can have subtle security issues depending on how encryption and MAC are combined.
- **D) MAC only — FALSE.** A MAC provides integrity and authenticity but no confidentiality. Without encryption, the plaintext remains exposed.

Q5. CRIME Attack Leakage

Why does compress-then-encrypt leak information?

- A) Ciphertext length correlates with compression efficiency.
- B) Identical prefixes compress better.
- C) Random IV eliminates the leak.
- D) Adaptive queries allow byte-wise secret recovery.

- **Correct options: A, B, D**
- **A) Ciphertext length correlates with compression efficiency — TRUE.** The slides show that when data is compressed before encryption, the resulting ciphertext length reflects how well the plaintext compresses. In the CRIME example, a second message that compresses better produces a shorter ciphertext, allowing an observer to distinguish messages based on length.
(Ref: 07-crypto hash)
- **B) Identical prefixes compress better — TRUE.** In the CRIME attack, the attacker crafts guesses that create repeated prefixes with the secret (e.g., cookies). Matching prefixes improve compression and therefore produce slightly shorter ciphertexts, enabling the attacker to infer correct guesses.
- **C) Random IV eliminates the leak — FALSE.** A random IV randomizes ciphertext contents but does not hide the ciphertext length. The information leak arises from length differences caused by compression, not from block equality or IV reuse.
- **D) Adaptive queries allow byte-wise secret recovery — TRUE.** The slides explicitly demonstrate that attackers can adaptively submit guesses and observe whether the ciphertext becomes slightly shorter, allowing recovery of secret data character by character.
(Ref: 07-crypto hash)

Q6. Merkle-Damgård Security Implications

Which are true about Merkle-Damgård hashes?

- A) Collision resistance of the compression function implies collision resistance of the hash.
- B) Length extension is inherent.
- C) Sponge construction eliminates length extension.
- D) Padding prevents collision attacks.

- **Correct options: A, B, C**
- **A) Collision resistance of compression $\Rightarrow$ collision resistance of hash — TRUE.** The slide states: “*Thm 1: if h is collision resistant then so is H* ”, indicating that the collision resistance of the compression function is preserved by the Merkle–Damgård construction.
(Ref: 07-crypto hash)
- **B) Length extension is inherent — TRUE.** In Merkle–Damgård, the final hash value exposes the internal chaining value, and the construction continues iterating the compression function from that state. This inherent property enables length extension attacks.
- **C) Sponge construction eliminates length extension — TRUE (in practice).** The slides introduce the sponge construction (used in SHA-3) as an alternative hash design that does not follow the Merkle–Damgård paradigm. Since the internal state is not directly exposed in the same way, the specific Merkle–Damgård length-extension property does not apply.
(Ref: 07-crypto hash)
- **D) Padding prevents collision attacks — FALSE.** Padding is necessary for correctness and unambiguous parsing in Merkle–Damgård, but it does not prevent collisions. Collision resistance depends on the security of the compression function and the birthday bound, not on padding alone.

Q7. Randomness Engineering

Which failures can weaken cryptographic key generation?

- A) Low entropy sources not mixed properly.
- B) Reusing internal PRNG state without reseeding.
- C) Using hardware RNG exclusively.
- D) Deterministic key derivation without entropy.

- **Correct options: A, B, D**
- **Why:** The randomness slide states that PRGs “*continuously add entropy to internal state*” and lists entropy sources such as hardware RNG instructions (e.g., `RdRand`) and timing interrupts (keyboard/mouse). This emphasizes the importance of entropy collection and proper mixing.
(Ref: 07-crypto hash)
- **A) Low entropy sources not mixed properly — TRUE.** If entropy sources are weak or not mixed correctly into the internal state, the resulting PRNG state can become predictable, directly weakening key generation.
- **B) Reusing PRNG state without reseeding — TRUE.** If the internal state is reused, leaked, or compromised and not refreshed with new entropy, subsequent outputs become predictable, enabling attackers to recover or predict generated keys.
- **C) Using hardware RNG exclusively — FALSE.** Hardware RNG can serve as a valid entropy source as discussed in the class slides. However, best practice is to mix multiple entropy sources for robustness.
- **D) Deterministic key derivation without entropy — TRUE.** If key generation is purely deterministic and does not incorporate fresh entropy, an attacker who knows the algorithm and inputs can reproduce the same keys, completely breaking security.

Q8. Public Key Trust Model

Which statements about PKI are correct?

- A) Certificates bind public keys to identities.
- B) PKI prevents all impersonation attacks automatically.
- C) Revocation is operationally difficult.
- D) Trust can be hierarchical or web-based.

• **Correct options: A, C, D**

- **Why:** The *PublicKey* slides describe the role of PKI in binding public keys to identities, different trust models (hierarchical trust and web-of-trust), and explicitly mention “*revocation inefficiencies*”.
(Ref: *PublicKey*)
- **A) Certificates bind public keys to identities — TRUE.** The slide explicitly states that a certificate binds a public key to an individual or entity, enabling others to verify the authenticity of the key.
- **B) PKI prevents all impersonation automatically — FALSE.** PKI does not eliminate all impersonation risks. Revocation delays, CA compromise, user warning bypass, and misconfiguration can still allow impersonation.
- **C) Revocation is operationally difficult — TRUE.** The slide literally states “*revocation inefficiencies*”, indicating that timely and reliable revocation remains a practical challenge.
- **D) Trust can be hierarchical or web-based — TRUE.** The slides explicitly describe both a *hierarchical trust model* and a *web-of-trust* model for establishing trust in public keys.

Q9. RSA Key Validity

Suppose $p = 17$, $q = 23$, and $\varphi(N) = 352$. Which public exponents e are valid for RSA?

- A) $e = 3$
- B) $e = 11$
- C) $e = 17$
- D) $e = 32$

• **Correct options: A, C**

• **Given:** $p = 17$, $q = 23$. Then

$$N = pq = 17 \cdot 23 = 391, \quad \varphi(N) = (p-1)(q-1) = 16 \cdot 22 = 352.$$

• **RSA validity condition:** A public exponent e is valid only if

$$\gcd(e, \varphi(N)) = 1,$$

i.e., e must be relatively prime to $\varphi(N)$.

(Ref: *PublicKey*)

• **Factor $\varphi(N)$:**

$$352 = 32 \cdot 11 = 2^5 \cdot 11.$$

So any e sharing a factor 2 or 11 with 352 is invalid.

• **Check options:**

- **A) $e = 3$:** $\gcd(3, 352) = 1 \Rightarrow$ **TRUE**. (352 is not divisible by 3.)
- **B) $e = 11$:** $\gcd(11, 352) = 11 \Rightarrow$ **FALSE**. (352 is divisible by 11.)
- **C) $e = 17$:** $\gcd(17, 352) = 1 \Rightarrow$ **TRUE**. (17 shares no common factor with $2^5 \cdot 11$.)
- **D) $e = 32$:** $\gcd(32, 352) = 32 \Rightarrow$ **FALSE**. (Both are divisible by 2^5 .)

Q10. Collision Probability (Birthday Bound)

A 256-bit hash function is used, and an attacker hashes $q = 2^{64}$ random messages.

Which statements are correct?

- A) Collision probability is approximately $\frac{q(q-1)}{2^{257}}$.
- B) Collision probability is approximately 2^{-64} .
- C) To obtain approximately 50% collision probability, $q \approx 2^{128}$.
- D) Hash output length should be roughly double the block cipher key length due to birthday attacks.

• **Correct options: A, C, D**

• **Birthday bound reminder:** For an n -bit hash and $q \ll 2^{n/2}$ random queries, the collision probability is approximated by

$$P(\text{collision}) \approx 1 - \exp\left(-\frac{q(q-1)}{2^{n+1}}\right) \approx \frac{q(q-1)}{2^{n+1}}.$$

• Here $n = 256$ and $q = 2^{64}$.

• **Option A.**

$$\frac{q(q-1)}{2^{n+1}} \approx \frac{(2^{64})^2}{2^{257}} = \frac{2^{128}}{2^{257}} = 2^{-129}.$$

Hence the collision probability is approximately 2^{-129} . **A is TRUE.**

• **Option B.** The claim is that the probability is approximately 2^{-64} , but the calculation above gives approximately 2^{-129} , which is vastly smaller. **B is FALSE.**

• **Option C.** A collision probability of roughly 50% occurs around the birthday threshold

$$q \approx 2^{n/2} = 2^{128}.$$

C is TRUE.

• **Option D.** The hash slides explicitly state: *“Because of the birthday attack, the length of hash outputs in general should double the key length of block ciphers.”*
(Ref: 07-crypto hash) **D is TRUE.**

Q11 Energy Depletion Attacks

Energy depletion attacks are a serious threat to battery-powered IoT devices. Which of the following design choices helps mitigate such attacks at the cryptographic or protocol level, and why?

- A) Encrypt all protocol messages using public-key cryptography so that attackers cannot inject traffic that triggers device processing.
- B) Limit the number of cryptographic handshakes or authentication attempts per time window to prevent repeated triggering of energy-expensive operations.
- C) Increase the battery capacity of the device to tolerate higher energy consumption during attacks.
- D) *Reject unauthenticated or unauthorised requests using a lightweight verification step before performing any expensive cryptographic operations or maintaining protocol state.*

- **Only D**, is the only correct answer. This lets us control the number of messages that actually get processed without expending a lot of energy. This directly addresses the root cause of battery-drain attacks: forcing constrained devices to perform expensive work. Early, low-cost rejection prevents attackers from exploiting computational asymmetry.
- Option A is incorrect. Encryption protects confidentiality and integrity, but does not prevent attackers from initiating protocol interactions. Moreover, public-key cryptography may itself increase energy consumption
- Option B is incorrect. Rate-limiting reduces the impact of repeated attacks and is a useful secondary defence, but it does not eliminate the cost of initial authentication attempts and can be bypassed by distributed attackers.
- Option C is incorrect. Increasing battery capacity is a hardware mitigation, not a cryptographic or protocol-level defence, and does not prevent the attack itself.

Q12 Factorization:

Observe the difference in difficulty of solving the following problems:

- Compute $x = 39119 * 53147$
- Compute factors of 2079057493

The first problem is trivial, while the second becomes computationally hard when the prime factors are very large. In the case above, two 16-bit prime numbers are being multiplied. Ron Rivest, Adi Shamir, and Leonard Adleman exploited this asymmetry to design the RSA public-key cryptosystem.

Do you agree with the following statements? Provide reasons:

- A) The security of RSA relies on the computational difficulty of factoring a large composite number into its prime factors.

- w • True, Knowledge of p and q allows computation of $\phi(n)$, which in turn enables the derivation of the private key d . Therefore, breaking RSA is believed to be equivalent to (or at least strongly related to) the integer factorisation problem.

- B) RSA provides confidentiality, but cannot be used for authentication or digital signatures.

- False, RSA can be used for both encryption and digital signatures. Digital signatures (authentication, integrity, non-repudiation): Private key signs, public key verifies.

- C) RSA becomes insecure if the private key is generated using small or predictable prime numbers, even if the public key is large.

- True, If the primes are too small, poorly generated, predictable, or share factors with other moduli, then the modulus n can be factored efficiently using known attacks (e.g., GCD-based attacks, low-entropy prime attacks).

- D) Even if two different key pairs share a prime number used in key generation, the security of encryption remains uncompromised.

- $n_1 = p * q_1$ and $n_2 = p * q_2$ where the prime p is reused. An attacker can compute: $\gcd(n_1, n_2) = p$. The greatest common divisor can be computed very efficiently, even for extremely large numbers.